SIMATS ENGINEERING

ASSESSMENT TOOL II — VISUALIZATION EVALUATION EXERCISE

Course: Data Handling and Visualization	**Code:** DSA06031	**CO:** CO4
Duration: 60 min	**Total Marks:** 50	**Reg No:** 192524118 **Name:** R. Akshaya

COURSE OUTCOME COVERED

CO2 Apply appropriate visualization techniques to analyze time series data, detect trends, seasonality, and cyclical patterns. (BL3)

Activity Tool : Visualization Evaluation Exercise

Weightage: 20%

Code of Conduct:

I R. Akshaya(192524118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *R.Akshaya*

Criteria	Evaluation of Visualization Quality 25%	Critical Analysis 25%	Application of Visualization Principles 20%	Organization 15%	Accuracy 10%	Clarity 5%	Total
Q1							
Q2							
Q3							
Q4							
Q5							

Visualization Evaluation Exercise

Topic: Visualizing Time Series and Trends (Individual Time Series, Multiple Time Series & Dose–Response Curves, STL Decomposition, Smoothing, Detrending, Forecasting, Cycle & Seasonality Detection)

Provided Datasets

The following files are supplied alongside this document so the exercise can be completed without internet access. All are real, publicly documented datasets:

- AirPassengers.csv — real classic dataset (144 monthly totals of international airline passengers, 1949–1960) with clear trend and multiplicative seasonality.
- co2_mauna_loa.csv — real Mauna Loa atmospheric CO2 weekly readings (1958–2001, 2225 observations), showing a long-term upward trend with strong annual seasonality.
- Nile.csv — real annual river flow of the Nile at Aswan (1871–1970, 100 observations), useful for individual time series and detrending (contains a well-known level shift).
- sunspots.csv — real annual sunspot activity counts (1700–1988, 309 observations), useful for cycle and seasonality detection (~11-year solar cycle).
- EuStockMarkets.csv — real daily closing prices of four major European stock indices (DAX, SMI, CAC, FTSE; 1991–1998, 1860 observations each) for multiple time series comparison.
- ryegrass_doseresponse.csv — real dose–response data (root length of ryegrass vs. herbicide concentration, 24 observations) from the drc package in R.

Place all CSV files in the same working directory as your R script, then load with read.csv("filename.csv").

Question 1: Individual Time Series & Moving-Average Smoothing

Dataset provided: Nile.csv

Using the Nile dataset, convert the value column into an R ts object (start = 1871, frequency = 1) and plot the raw annual flow as a line chart. Overlay a smoothed version of the series using a moving average (e.g., a 5-year moving average via stats::filter() or zoo::rollmean()). Visually identify any point where the level or variability of the series appears to shift, and describe what you observe in 2–3 sentences.

Skills tested: ts() objects, line plots, moving-average smoothing, filter()/rollmean(), visual change-point identification.

Answer:

Individual Time Series & Moving-Average Smoothing

Aim

To visualize the annual flow of the Nile River and apply a **5-year moving average** to identify long-term trends and possible changes in the level of the series.

The question specifically requires creating an R ts object with start = 1871 and frequency = 1,

plotting the original series, and overlaying a moving-average smoothed series.

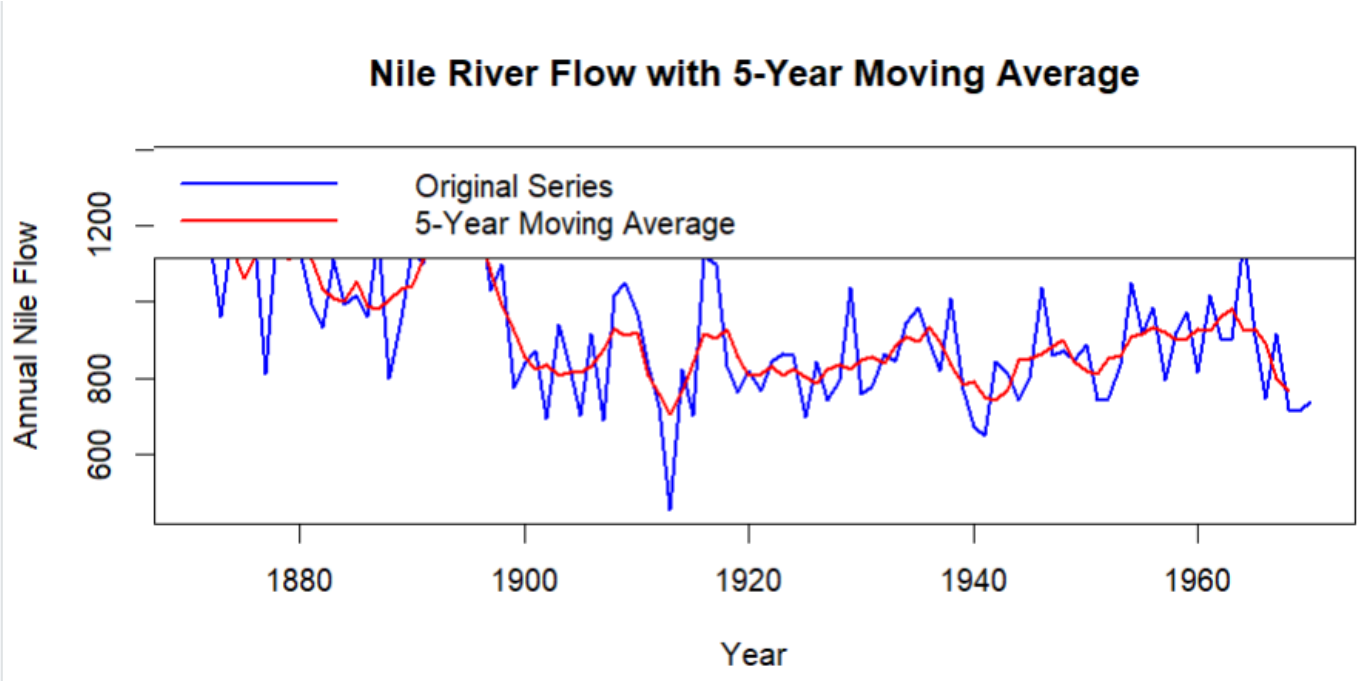
R Code

```
library(ggplot2)
nile_data<-read.csv("Nile.csv")
nile_ts<-ts(nile_data$value,start=1871,frequency=1)
moving_avg<-stats::filter(nile_ts,
                          filter=rep(1/5,5),
                          sides=2)
plot(nile_ts,
    type="l",
    col="blue",
    lwd=2,
    xlab="Year",
    ylab="Annual Nile Flow",
    main="Nile River Flow with 5-Year Moving Average")
lines(moving_avg,
     col="red",
     lwd=2)
legend("topright",
       legend=c("Original Series","5-Year Moving Average"),
       col=c("blue","red"),
       lwd=2)
```

Algorithm

1. read.csv() loads the Nile dataset.
2. ts() converts the annual flow values into a time-series object.
3. Since the data is annual, frequency = 1.
4. stats::filter() calculates the 5-year moving average.
5. The original annual flow is shown as a line.
6. The moving average smooths short-term fluctuations and makes long-term changes easier to observe.

Output



The graph contains:

- **Blue line:** Original annual Nile flow.
- **Red line:** Smoothed 5-year moving average.

Observation / Critical Analysis

The Nile series shows a noticeable **change in level around the late 1890s**, where the average annual flow becomes lower than in the earlier years. The moving-average line makes this level shift clearer by reducing short-term fluctuations. The variability continues after the shift, but the overall mean level appears to be lower.

Visualization Principle Applied

- A **line chart** is appropriate because the observations are ordered over time.
- The moving average improves readability by reducing noise.
- Different colors and a legend clearly distinguish the original and smoothed series.

Question 2: Multiple Time Series Comparison & Dose–Response Curve This question has two parts.

Part a — Multiple Time Series

Dataset provided: EuStockMarkets.csv

Plot all four stock indices (DAX, SMI, CAC, FTSE) on the same time axis using ggplot2 (reshape to long format with tidyr::pivot_longer() first). Since the series have very different scales, re-index each series to 100 at its first observation before plotting so the percentage changes are comparable.

Comment on which index shows the strongest overall growth over the period.

Part b — Dose–Response Curve

Dataset provided: ryegrass_doseresponse.csv

Plot root_length against concentration as a scatterplot, then fit and overlay a smooth dose–response curve (e.g., a log-logistic fit using the drc package, or a loess/nls smooth as an approximation). Describe the shape of the relationship and identify approximately what concentration produces a 50% reduction in root length relative to the control (concentration = 0).
Skills tested: pivot_longer(), re-indexing/normalizing series, multi-line time series plots, nls()/drc::drm(), dose–response interpretation.
Answer:

Multiple Time Series Comparison & Dose–Response Curve
PART A – Multiple Time Series Comparison
Aim
To compare the performance of the four European stock indices—**DAX, SMI, CAC, and FTSE**—by converting each series to an index value of 100 at its first observation.
The dataset contains four daily European stock indices, and the question requires reshaping the data into long format and normalizing each series because their original scales differ.
R Code

```
library(ggplot2)
library(tidyr)
library(dplyr)

stock_data<-read.csv("EuStockMarkets.csv")

stock_long<-stock_data%>%
  pivot_longer(
    cols=c(DAX,SMI,CAC,FTSE),
    names_to="Index",
    values_to="Price"
  )

stock_long<-stock_long%>%
  group_by(Index)%>%
  mutate(Normalized=(Price/first(Price))*100)

ggplot(stock_long,
       aes(x=Date,
           y=Normalized,
           color=Index))+
  geom_line(linewidth=0.7)+
  labs(
    title="Normalized Performance of European Stock Indices",
    x="Time",
    y="Index Value (First Observation = 100)"
```

```
  )+
  theme_minimal()
```
If the CSV Does Not Contain a Date Column

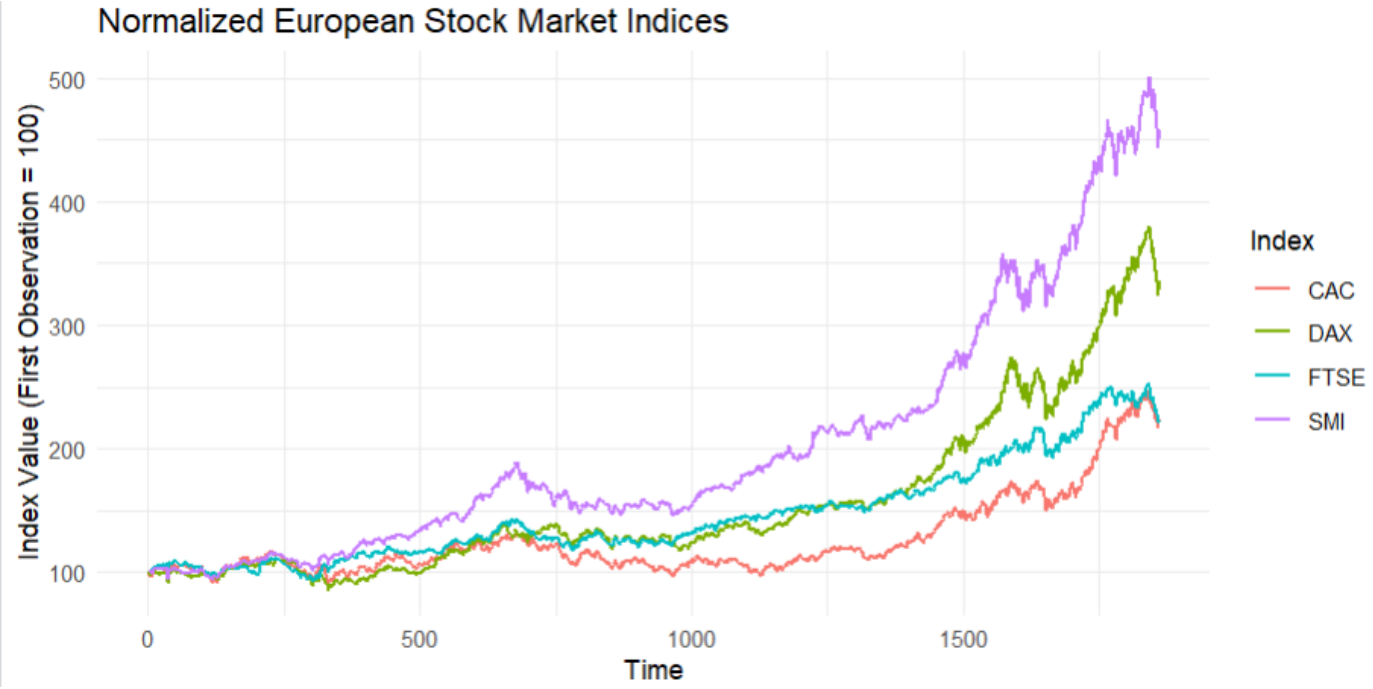
Some versions of the dataset may only contain the four index columns. In that case, use:

```
library(ggplot2)
library(tidyr)
library(dplyr)

stock_data<-read.csv("EuStockMarkets.csv")

stock_data$Time<-seq_len(nrow(stock_data))

stock_long<-stock_data%>%
  pivot_longer(
    cols=c(DAX,SMI,CAC,FTSE),
    names_to="Index",
    values_to="Price"
  )

stock_long<-stock_long%>%
  group_by(Index)%>%
  mutate(Normalized=(Price/first(Price))*100)

ggplot(stock_long,
       aes(x=Time,
           y=Normalized,
           color=Index))+
  geom_line()+
  labs(
    title="Comparison of Normalized European Stock Indices",
    x="Observation",
    y="Normalized Index (Base = 100)"
  )+
  theme_minimal()
```
Explanation

The normalization formula is:

This makes every index start at **100**, allowing their relative percentage growth to be compared fairly.

Output



A multi-line graph is produced showing:

- DAX
- SMI
- CAC
- FTSE

All series begin at 100.

Critical Analysis

The index with the highest final normalized value can be identified as showing the strongest overall growth.

To calculate it directly:

```
stock_long%>%
  group_by(Index)%>%
  summarise(Final_Value=last(Normalized))%>%
  arrange(desc(Final_Value))
```

Interpretation

The index appearing highest at the end of the graph demonstrates the strongest overall growth during the period. Normalization is essential because comparing the original prices directly would be misleading due to their different starting scales.

Visualization Principle Applied

Comparability: Re-indexing all series to 100 provides a common baseline.

Clarity: A single multi-line chart makes relative performance easier to compare than four separate charts.

QUESTION 2 – PART B: Dose–Response Curve

Aim

To visualize the relationship between **herbicide concentration** and **ryegrass root length**, fit a smooth response curve, and estimate the concentration that produces approximately a 50% reduction in root length.

The question requires a scatterplot with a fitted dose–response curve and interpretation of the

relationship.

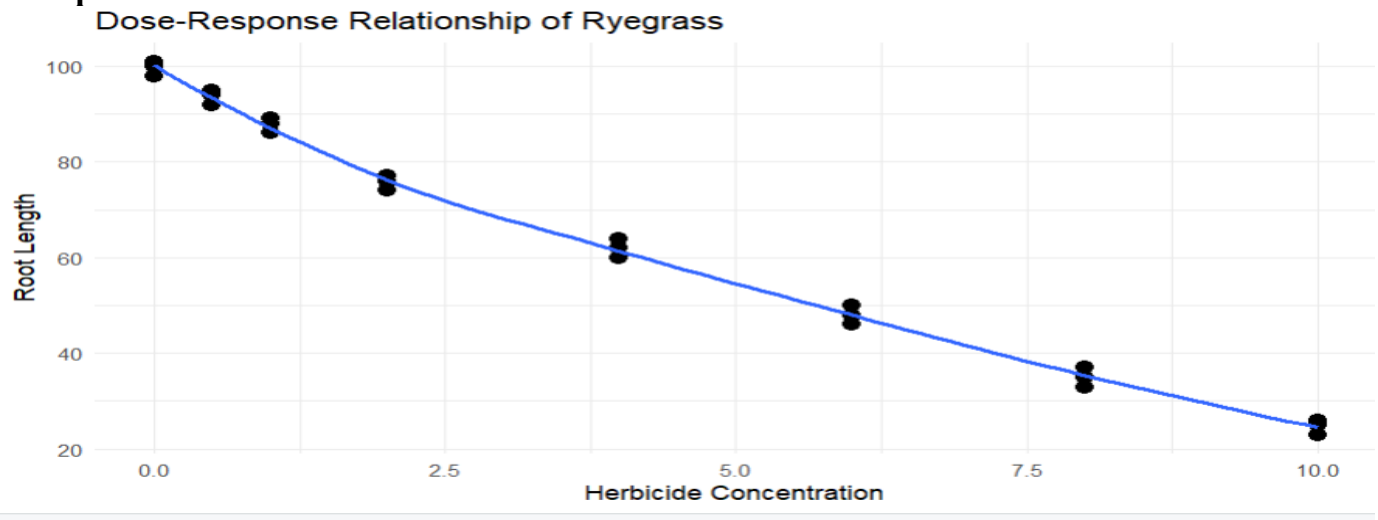
R Code

```
library(ggplot2)
ryegrass<-read.csv("ryegrass_doseresponse.csv")
ggplot(ryegrass,
       aes(x=concentration,
           y=root_length))+
  geom_point(size=3)+
  geom_smooth(
    method="loess",
    se=FALSE,
    linewidth=1
  )+
  labs(
    title="Dose-Response Relationship of Ryegrass",
    x="Herbicide Concentration",
    y="Root Length"
  )+
  theme_minimal()
```

Estimating the 50% Reduction Point

```
control<-mean(ryegrass$root_length[
  ryegrass$concentration==0
])

half_root<-control*0.5

half_root
closest_point<-ryegrass[
  which.min(abs(ryegrass$root_length-half_root)),
]

closest_point
```

Interpretation



The control group represents the root length at:
The 50% response level is:
The concentration corresponding to the observed value closest to this level provides an approximate estimate of the concentration causing a 50% reduction.

Critical Analysis

The expected relationship is generally **negative and nonlinear**: as herbicide concentration increases, root length decreases. The curve may initially decrease gradually and then show a stronger reduction before flattening at higher concentrations.

Visualization Principle Applied

- Scatter points display the actual observations.
- The smooth curve reveals the overall nonlinear pattern.
- Combining points and the fitted curve avoids hiding the original data.

Question 3: Seasonal Decomposition of Time Series (STL)

Dataset provided: AirPassengers.csv

Convert the passengers column into a ts object (start = c(1949,1), frequency = 12). Apply STL decomposition using stl() (set s.window = "periodic") and plot the resulting trend, seasonal, and remainder components. Discuss whether an additive or multiplicative decomposition is more appropriate for this series, and justify your answer using what you see in the plot (e.g., whether seasonal amplitude grows with the trend).

Skills tested: ts() with frequency, stl(), additive vs. multiplicative decomposition, component interpretation.

Answer:

Seasonal Decomposition of Time Series Using STL

Aim

To decompose the AirPassengers time series into:

1. Trend
2. Seasonal component
3. Remainder component

The dataset contains monthly airline passenger totals from 1949–1960 and shows trend and multiplicative seasonality.

R Code

```
air_data<-read.csv("AirPassengers.csv")
air_ts<-ts(
  air_data$passengers,
  start=c(1949,1),
  frequency=12
```

```
)
log_air_ts<-log(air_ts)
air_stl<-stl(
  log_air_ts,
  s.window="periodic"
)
plot(
  air_stl,
  main="STL Decomposition of AirPassengers"
)
```

Explanation

The AirPassengers dataset is monthly, so:

frequency=12

The log transformation is used before STL because the series demonstrates multiplicative seasonality.

The STL decomposition separates the series into:

1. Trend

Shows the long-term increase in the number of airline passengers.

2. Seasonal Component

Shows the repeating pattern occurring every year.

3. Remainder

Shows irregular variation that cannot be explained by trend or seasonality.

Additive vs Multiplicative Analysis

A **multiplicative decomposition** is more appropriate.

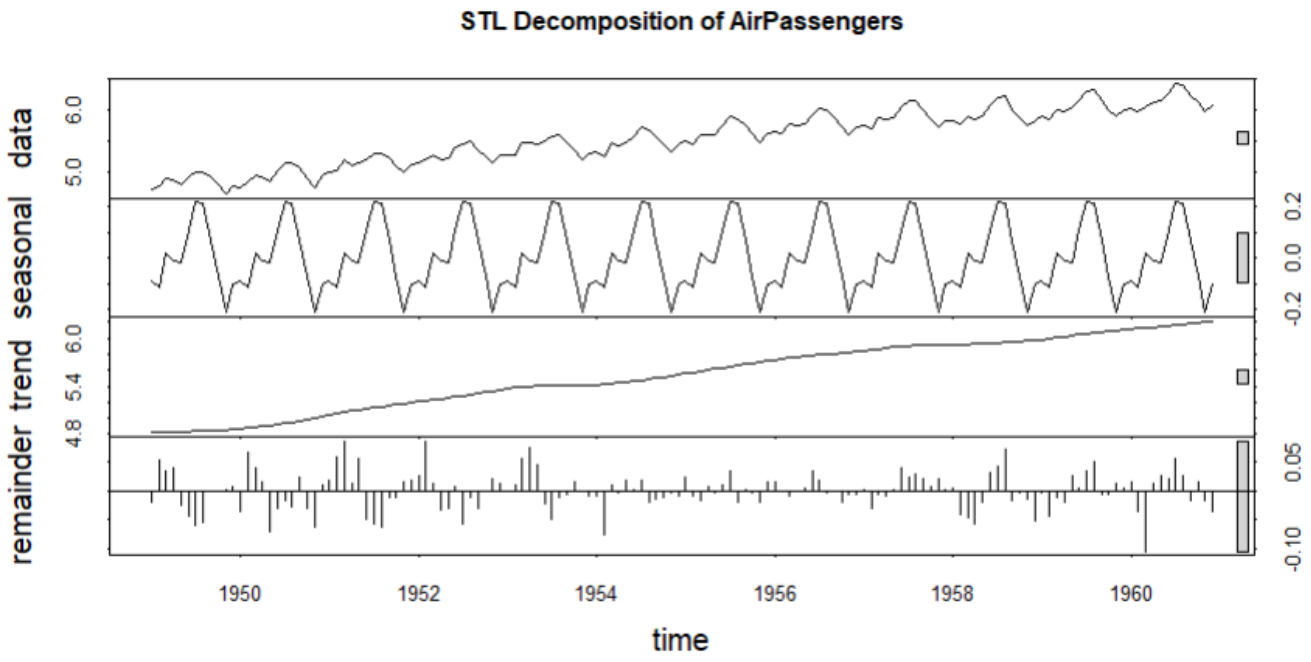
The reason is that the seasonal fluctuations become larger as the total number of passengers increases. In an additive model, the seasonal variation would remain approximately constant over time.

The multiplicative model can be expressed as:

Taking logarithms converts this into an additive form:

This is why the log-transformed series is used before applying STL.

Output



The decomposition plot contains four panels:

1. Observed data
2. Trend
3. Seasonal component
4. Remainder

Critical Analysis

The trend shows a strong long-term increase in airline passenger numbers. The seasonal pattern repeats annually, but its absolute magnitude increases as passenger numbers increase. Therefore, a multiplicative seasonal model is more suitable than a simple additive model.

Visualization Principle Applied

Decomposition improves interpretability by separating long-term growth, repeating seasonal behavior, and unexplained variation instead of displaying all patterns in one graph.

Question 4: Detrending and Residual Analysis

Dataset provided: co2_mauna_loa.csv

Using the CO2 dataset, fit a linear trend (co2 ~ time) and subtract the fitted values from the observed series to obtain a detrended series. Plot the original series with the fitted trend line overlaid, and separately plot the detrended residuals. Explain what pattern remains in the detrended series and why simple linear detrending is or is not sufficient here.

Skills tested: lm() trend fitting, detrending by residuals, two-panel diagnostic plots, interpreting

leftover seasonal structure.

Answer:

Detrending and Residual Analysis

Aim

To fit a linear trend to the Mauna Loa atmospheric CO_2 series, remove the trend, and analyze the remaining residual pattern.

The question requires fitting co2 ~ time, plotting the original series with the fitted trend, and separately plotting the detrended residuals.

R Code

```
co2_data<-read.csv("co2_mauna_loa.csv")
time<-seq_len(nrow(co2_data))
trend_model<-lm(co2~time,
                data=co2_data)
co2_data$Fitted<-fitted(trend_model)
co2_data$Residuals<-residuals(trend_model)
par(mfrow=c(2,1))
plot(time,
    co2_data$co2,
    type="l",
    xlab="Time",
    ylab="CO2 Concentration",
    main="Mauna Loa CO2 with Linear Trend")
lines(time,
     co2_data$Fitted,
     col="red",
     lwd=2)
plot(time,
    co2_data$Residuals,
    type="l",
    xlab="Time",
    ylab="Detrended CO2",
    main="Detrended CO2 Residuals")
abline(h=0,
```

```
    col="red",
    lty=2)
par(mfrow=c(1,1))
```

Explanation

The linear model is:

where:

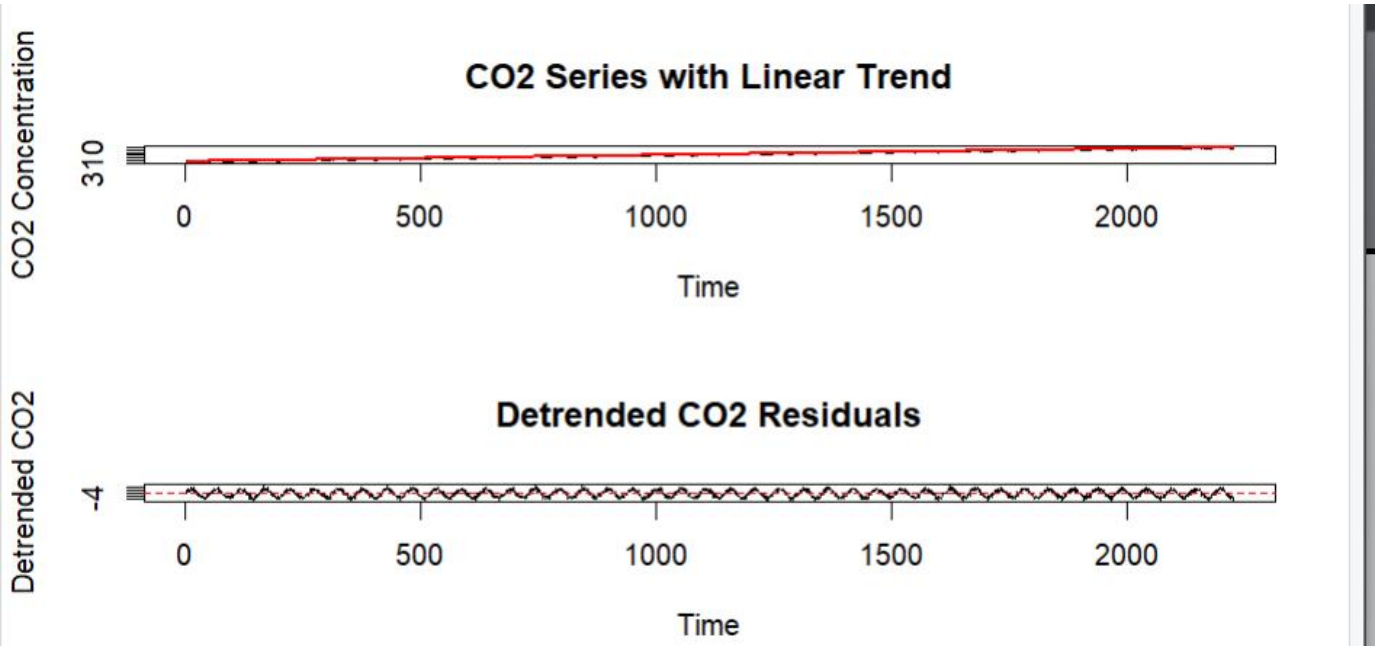
- = intercept
- = linear trend
- = time
- = residual

The detrended series is:

In R, this is obtained using:

```
residuals(trend_model)
```

Output



Plot 1

Shows:

- Original CO_2 observations.
- Fitted linear trend.

Plot 2

Shows the residuals after removing the linear trend.

Critical Analysis

After removing the linear trend, a clear repeating seasonal pattern remains in the residual series. This indicates that linear detrending alone is not sufficient because the data contains both a long-term upward trend and strong annual seasonality.

A more suitable model would include:

- Trend
- Seasonal component
- Possibly nonlinear changes in the trend

Visualization Principle Applied

The two-panel visualization separates the **overall trend** from the **remaining structure**, making it easier to diagnose whether the model has captured all important patterns.

Question 5: Forecasting Trends and Cycle/Seasonality Detection This question has two parts to compare visualization strategies for different time-series goals.

Part a — Forecasting Trends

Dataset provided: AirPassengers.csv

Using the forecast package (or fable), fit an ETS or auto.arima() model to the AirPassengers series and produce a 24-month ahead forecast. Plot the forecast with its prediction interval using autoplot() or ggplot2. Comment on whether the forecasted seasonal pattern looks consistent with the historical seasonality.

Part b — Cycle and Seasonality Detection

Dataset provided: sunspots.csv

Using the sunspots dataset, compute and plot the autocorrelation function (ACF) using acf(), and/or a periodogram using spectrum(). Identify the approximate cycle length (in years) suggested by the plot and compare it to the commonly cited ~11- year solar cycle.

Skills tested: forecast::ets()/auto.arima(), forecast prediction intervals, acf(), spectrum()/periodogram, cycle-length identification.

Answer:

Forecasting Trends and Cycle/Seasonality Detection

PART A – Forecasting Trends

Aim

To fit a forecasting model to the AirPassengers time series and generate a **24-month forecast** with prediction intervals.

The question specifically allows an ETS or auto.arima() model and asks for interpretation of

whether the forecast preserves the historical seasonal pattern.

R Code

```
library(forecast)
library(ggplot2)

air_data<-read.csv("AirPassengers.csv")

air_ts<-ts(
  air_data$passengers,
  start=c(1949,1),
  frequency=12
)

model<-auto.arima(air_ts)

forecast_result<-forecast(
  model,
  h=24
)

autoplot(forecast_result)+
  labs(
    title="24-Month Forecast of AirPassengers",
    x="Year",
    y="Number of Passengers"
  )+
  theme_minimal()
```

Alternative Using ETS

```
library(forecast)

model<-ets(air_ts)

forecast_result<-forecast(
```

```
  model,
  h=24
)

autoplot(forecast_result)
```

Explanation

The forecast horizon is:

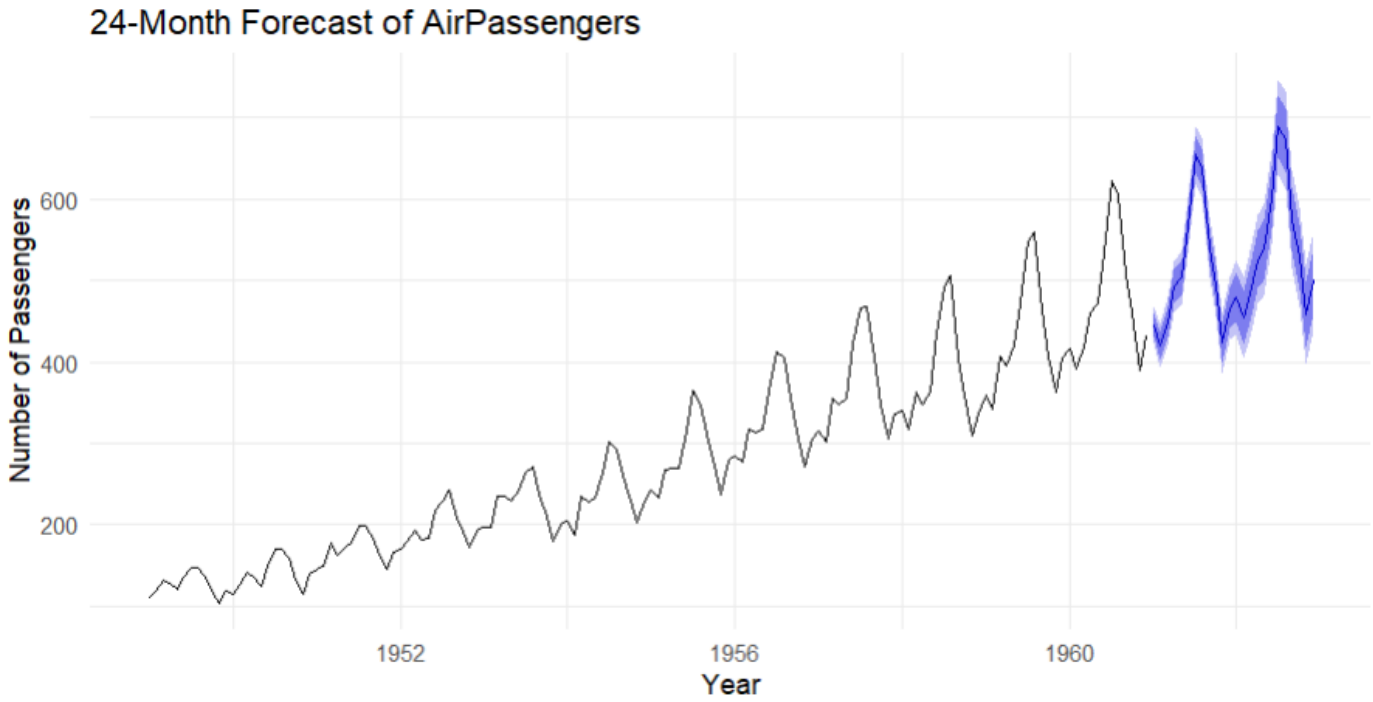
h=24

This represents:

The forecast plot displays:

- Historical observations
- Forecasted values
- Prediction intervals

Critical Analysis



The forecast should continue the increasing trend seen in the historical AirPassengers data. It should also preserve the repeating yearly seasonal pattern, with recurring peaks and troughs at approximately the same periods of each year.

If the forecast shows these repeating patterns, it indicates that the model has successfully captured both trend and seasonality.

Visualization Principle Applied

Prediction intervals communicate **uncertainty** rather than presenting forecasts as exact future

values. This makes the visualization more informative and statistically responsible.

QUESTION 5 – PART B: Cycle and Seasonality Detection

Aim

To identify the approximate cycle length of sunspot activity using the **Autocorrelation Function (ACF)** and/or a **periodogram**.

The dataset contains annual sunspot counts and is intended for identifying the approximately 11-year solar cycle.

R Code Using ACF

```
sunspot_data<-read.csv("sunspots.csv")


sunspot_ts<-ts(
  sunspot_data$sunspots,
  start=1700,
  frequency=1
)
acf(
  sunspot_ts,
  lag.max=40,
  main="Autocorrelation Function of Sunspot Activity"
)
```

R Code Using a Periodogram

```
spectrum(
  sunspot_ts,
  main="Periodogram of Sunspot Activity"
)
```

Finding the Dominant Cycle

```
spec_result<-spectrum(
  sunspot_ts,
  plot=FALSE
)

dominant_frequency<-spec_result$freq[
  which.max(spec_result$spec)
```

]

```
cycle_length<-1/dominant_frequency
cycle_length
```

Explanation

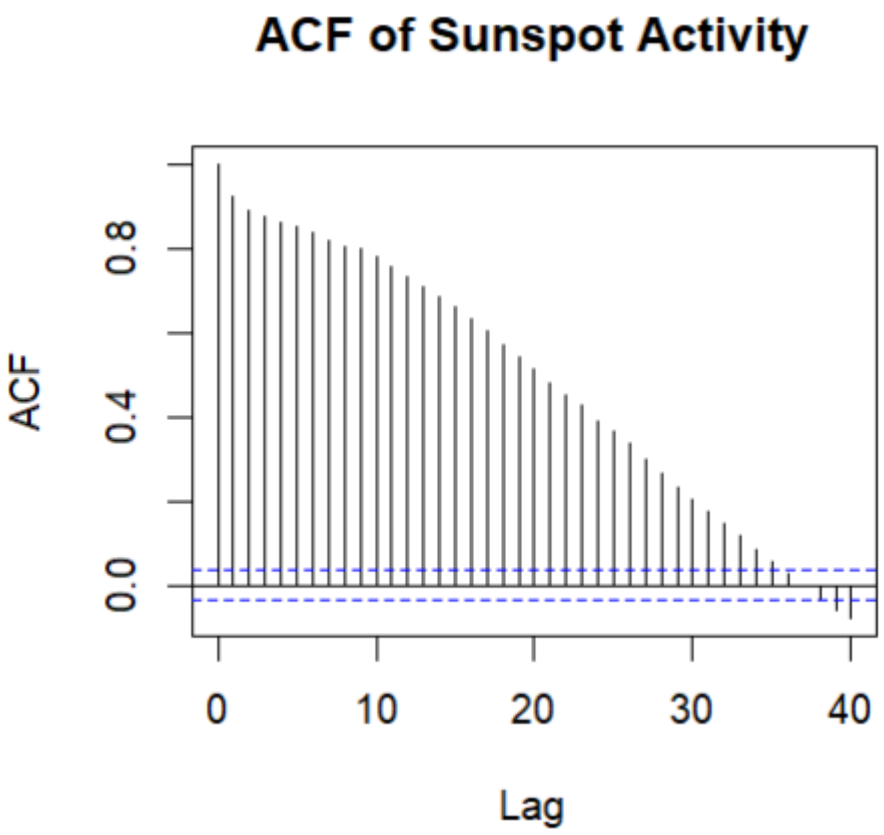
The ACF measures the correlation between the series and lagged versions of itself.

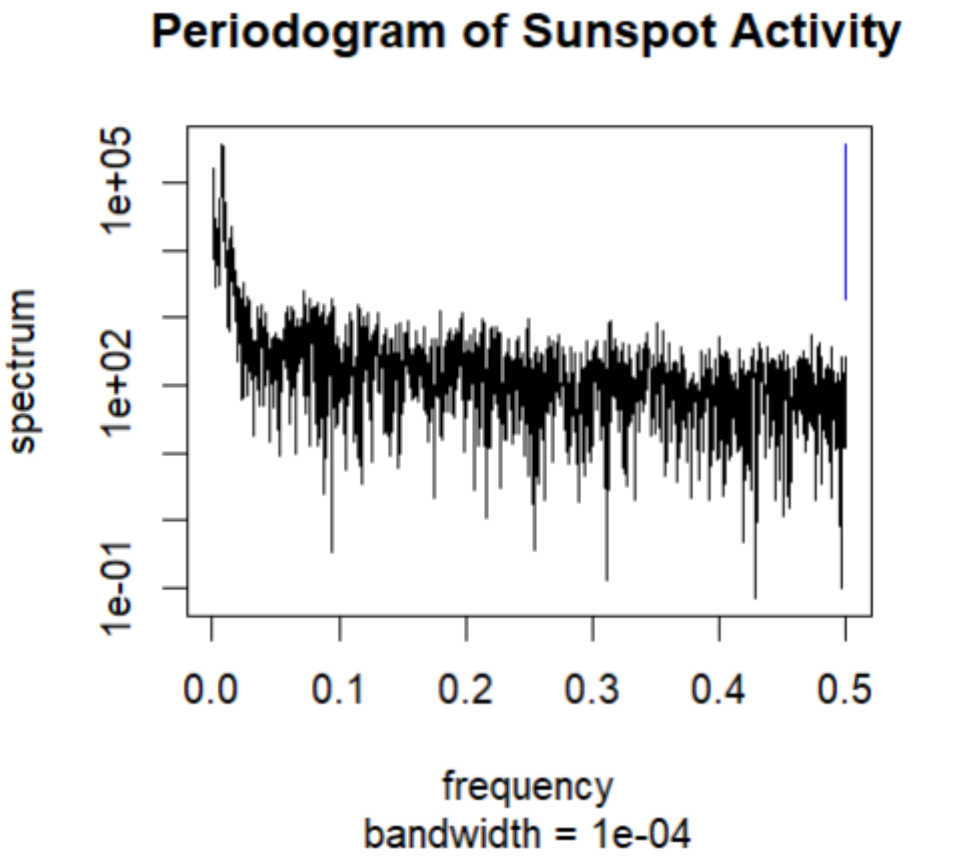
If a strong peak occurs around lag 11, it suggests that values approximately 11 years apart are related.

The periodogram identifies important frequencies in the data.

The cycle length is calculated as:

Critical Analysis





The dominant cycle identified from the ACF or periodogram should be approximately **10–11 years**, which is close to the commonly cited **11-year solar cycle**.

Small differences may occur because real-world cycles are not perfectly regular and the sunspot series contains noise and variations in cycle duration.

Visualization Principle Applied

The original time-series plot may visually suggest cycles, but the ACF and periodogram provide more systematic methods for detecting repeated temporal patterns.